

GHSTmarket

Propozycja projektu zaliczeniowego

1. Opis projektu

GHSTmarket to marketplace do sprzedaży i pobierania cyfrowych zasobów twórczych — inspirowany platformą Itch.io, ale z nowoczesnym, intuicyjnym interfejsem użytkownika. Platforma umożliwia twórcom publikowanie swoich prac (grafiki, szablony, fonty, muzykę, pliki gier indie) oraz ich wyszukiwanie, kupowanie i pobieranie przez innych użytkowników.

Projekt adresuje realny problem: istniejące platformy tego typu posiadają przestarzały i nieintuicyjny interfejs, utrudniający nawigację oraz odkrywanie nowych zasobów. GHSTmarket oferuje symulowany system portfela — użytkownicy mogą zasilać swoje konto wirtualnymi środkami oraz kupować zasoby innych twórców.

2. Struktura bazy danych

Baza danych składa się z 6 tabel powiązanych więzami klucza obcego, w tym jedną relacją wiele-do-wielu:

Tabela	Opis	Relacje
users	Konta użytkowników — dane profilu, saldo portfela, flaga is_admin	hasMany: products, purchases
products	Cyfrowe zasoby wystawione na sprzedaż lub bezpłatnie	belongsTo: users, categories; hasMany: purchases; belongsToMany: tags
categories	Kategorie zasobów (grafika, font, muzyka itd.)	hasMany: products
purchases	Historia zakupów — powiązanie użytkownika z zakupionym produktem	belongsTo: users, products
tags	Tagi opisujące zasoby	belongsToMany: products
product_tag	Tabela pośrednia relacji wiele-do-wielu	pivot: products ↔ tags

3. Planowane funkcjonalności

3.1 Operacje CRUD

Pełen CRUD dla każdego modelu:

- Users — rejestracja nowego konta, logowanie, edycja profilu, dezaktywacja konta (soft delete przez flagę `is_active`)
- Products — dodawanie zasobu z uploadem pliku, edycja, przeglądanie, usuwanie
- Categories — dodawanie, edycja i usuwanie kategorii (wyłącznie administrator)
- Tags — dodawanie, edycja i usuwanie tagów (użytkownik); przypisywanie tagów do własnych produktów
- Purchases — zapis zakupu przy transakcji; podgląd zakupionych produktów w zakładce "Moje zakupy" z możliwością pobrania pliku w dowolnym momencie (brak edycji i usuwania — celowe, zakup jest operacją nieodwracalną)

3.2 Wyszukiwanie

- Wyszukiwanie produktów po nazwie, opisie, tagu i kategorii
- Filtrowanie po cenie (bezpłatne / płatne), dacie dodania, popularności
- Wyszukiwanie użytkowników/twórców po nazwie

3.3 Walidacja danych

Walidacja obejmuje wymagane pola oraz następujące reguły biznesowe:

- Cena produktu nie może być ujemna
- Tytuł produktu: minimum 3, maksimum 100 znaków
- Opis produktu: minimum 20, maksimum 2000 znaków
- Adres e-mail użytkownika musi być unikalny i w poprawnym formacie
- Hasło: minimum 8 znaków, wymagana co najmniej jedna cyfra
- Upload pliku: tylko dozwolone rozszerzenia (zip, png, jpg, mp3, ttf, pdf), maksymalny rozmiar 50 MB
- Kwota zasilenia portfela musi być większa od zera
- Zakup niemożliwy gdy saldo portfela jest niewystarczające
- Produkt musi mieć przypisane co najmniej 1 i maksymalnie 5 tagów

3.4 Wzorzec MVC

Projekt implementuje wzorzec MVC zgodnie z konwencjami Laravela: modele Eloquent z relacjami, widoki Blade z layoutem i komponentami UI oraz warstwę sterującą w postaci kontrolerów HTTP (np. pobieranie plików, listy z query params) oraz komponentów Livewire (formularze, filtry, operacje CRUD w interfejsie). Walidacja wydzielana jest do Form Requestów lub reguł w komponentach Livewire.

3.5 Mechanizmy HTTP

- Query params — filtrowanie i sortowanie listy produktów (`?category=fonts&sort=newest&free=1`)
- URL params — szczegóły produktu i profil twórcy (`/products/{id}`, `/users/{id}`)
- Formularze POST — rejestracja, logowanie, dodawanie produktów, zasilenie portfela, zakup

3.6 Autentykacja

- Rejestracja nowego konta użytkownika z walidacją danych
- Logowanie i wylogowywanie
- Hashowanie haseł z użyciem bcrypt (Laravel Hash facade)
- Middleware chroniące trasy wymagające zalogowania
- Rozróżnienie ról: użytkownik oraz administrator (flaga is_admin w tabeli users)
- Autentykacja zaimplementowana jest z użyciem pakietu Laravel Fortify dostarczonego w starter kicie (rejestracja, logowanie, reset hasła, zarządzanie profilem).

3.7 System portfela (symulacja płatności)

Każdy użytkownik posiada wirtualny portfel z saldem. System symuluje rzeczywisty obieg pieniędzy:

- Zasilenie konta — użytkownik wprowadza kwotę, saldo rośnie (symulacja przelewu)
- Zakup zasobu — saldo kupującego maleje, saldo sprzedającego rośnie o cenę produktu

3.8 Upload i pobieranie plików

- Twórca uploaduje plik zasobu przy dodawaniu produktu (Laravel Storage)
- Plik przechowywany na serwerze poza katalogiem publicznym (storage/app/private)
- Pobranie pliku dostępne wyłącznie po zakupie — chroniony link generowany przez kontroler
- Próba pobrania bez zakupu zwraca błąd 403

3.9 Dodatkowe mechanizmy Laravel

- Eloquent relationships — hasMany, belongsTo, belongsToMany z tabelą pivot
- Form Requests — walidacja wydzielona do osobnych klas
- Seeders i Factories — dane testowe generowane automatycznie
- Pagination — stronicowanie listy produktów
- Laravel Storage — zarządzanie plikami z bezpiecznym dostępem

4. Technologie

Warstwa	Technologia
Backend / Framework	Laravel 13 (PHP 8.4+)
Baza danych	PostgreSQL
Widoki	Blade Templates
Reaktywność UI	Livewire v4

Warstwa	Technologia
Komponenty UI	Flux UI v2
Interakcje klienta	Alpine.js
Stylowanie	Tailwind CSS
Starter Kit	Laravel Livewire Starter Kit
ORM	Eloquent
Przechowywanie plików	Laravel Storage (local driver)

5. Uzasadnienie wyboru technologii

Jako punkt startowy projektu wykorzystano oficjalny Laravel Livewire Starter Kit, rekomendowany w dokumentacji Laravel 13 jako preferowane podejście do budowania aplikacji server-side z nowoczesnym interfejsem. Starter kit dostarcza gotową infrastrukturę autentykacji (rejestracja, logowanie, zarządzanie profilem) w postaci edytowalnego kodu — akcji Fortify, widoków Blade, komponentów Livewire i tras — co pozwoliło zaoszczędzić czas i skupić się na logice biznesowej aplikacji.

Wybór Livewire v4 jako warstwy reaktywności był świadomą decyzją architektoniczną. Livewire umożliwia budowanie dynamicznych interfejsów (modalne formularze, live search, dynamiczne filtry) bez konieczności pisania osobnego API czy przełączania się między PHP a JavaScript. Komponenty Livewire działają server-side i komunikują się z przeglądarką przez AJAX, zachowując jednocześnie pełną integrację z Eloquentem i pozostałymi mechanizmami Laravel.

Alpine.js — dołączony do Livewire Starter Kita — odpowiada za lekkie interakcje po stronie klienta: obsługę dropdownów, przełączanie widoczności elementów czy animacje UI. Jest to minimalistyczna alternatywa dla Vue czy React w sytuacjach, gdzie nie jest potrzebna pełna reaktywność — co doskonale sprawdza się w połączeniu z Livewire.

Flux UI v2 to biblioteka komponentów UI dołączona do Livewire Starter Kita — dostarcza gotowe, ostylowane Tailwindem komponenty (modale, formularze, dropdown, tabele) zintegrowane z Livewire. Dzięki temu możliwe jest budowanie spójnego, profesjonalnego interfejsu bez pisania powtarzalnego kodu UI od zera, co znacząco przyspiesza development przy zachowaniu wysokiej jakości wizualnej aplikacji.

PostgreSQL został wybrany jako silnik bazodanowy ze względu na lepszą obsługę typów danych, wydajność przy złożonych zapytaniach oraz pełną zgodność z Eloquent ORM — Laravel obsługuje PostgreSQL natywnie poprzez konfigurację sterownika w pliku `.env`.